

23.13 Fast Matrix Multiplication

Every algorithm in this chapter is built out of matrix products, and every cost we have counted has been counted against them. Section 12 broke one such count: the Fourier matrix is applied in about $n \log n$ operations, although the definition of a matrix–vector product asks for n^2 . The definition of a matrix product asks for n^3 multiplications. This section asks whether n^3 is necessary, answers no, and then asks what the answer costs in accuracy.

23.13.1 What a fast algorithm would be

Count the operations that Definition 0.9.9 prescribes. The product $\mathbf{AB}$ of two matrices in $M_n(F)$ has n^2 entries, each of them a sum of n products, so the recipe performs n^3 multiplications and $n^2(n-1)$ additions:

$$n^3 + n^2(n-1) = 2n^3 - n^2$$

arithmetic operations in all. Chapter 2 §06 made the same kind of count for an LU factorization and got $\frac{2}{3}n^3$, so a product costs about three times a factorization. Nothing so far has suggested that either number could be smaller.

Before asking whether it can be, we need to say what “it” is. A claim that some procedure multiplies matrices in fewer than n^3 multiplications is empty until the permitted procedures are fixed, and the fixing has to be done carefully: a procedure allowed to compare two numbers, or to branch on whether an entry is zero, is a different object from one that only computes.

A program is a list of arithmetic instructions, each using the inputs or earlier results, and its cost is the length of the list.

Definition 23.13.1. Arithmetic cost of a matrix product

Fix a field F and $n \geq 1$, and treat the $2n^2$ entries of $\mathbf{A}, \mathbf{B} \in M_n(F)$ as **indeterminates**. A **straight-line program** is a finite list $g_1, \dots, g_N$ in which each g_t is either

- (i) one of the $2n^2$ indeterminates, or a constant of F (an **input step**), or
- (ii) $g_r + g_s$, $g_r - g_s$ or $g_r g_s$ for some $r, s < t$ (an **arithmetic step**).

Each g_t is then a polynomial in the indeterminates. The program **computes the product** if for every i, j the polynomial $\sum_k a_{ik} b_{kj}$ occurs among the g_t . Its **cost** is the number of arithmetic steps, and

$$M(n) := \min \{ \text{cost of a program computing the } n \times n \text{ product} \}$$

is the **arithmetic cost** of the $n \times n$ product over F .

In words: input steps are free, because they only name the data; the three arithmetic operations each cost 1; and the program must produce all n^2 entries as **polynomial identities**, not merely for the particular numbers at hand. There are no comparisons, no branches and no loops — a straight-line program does the same thing whatever the data are. Division is excluded, which costs nothing here: none of the algorithms below uses it.

Two examples fix the definition. For $n = 1$ the single instruction $g_3 = g_1 g_2$ computes $a_{11} b_{11}$, so $M(1) = 1$. For $n = 2$ the recipe of Definition 0.9.9 is the program that forms the eight products $a_{ik} b_{kj}$ and then four sums, at a cost of 12; so $M(2) \leq 12$. A degenerate case worth naming: a program that lists only input steps has cost 0 and computes nothing, since no entry of $\mathbf{AB}$ is

an indeterminate.

The exponent of matrix multiplication is the smallest growth rate a family of such programs can have.

Definition 23.13.2. The exponent of matrix multiplication

The **exponent of matrix multiplication** over F is

$$\omega := \inf \{ \tau \in \mathbb{R} : M(n) = O(n^\tau) \text{ as } n \rightarrow \infty \}.$$

The infimum is over **all** real τ for which some constant c and some n_0 give $M(n) \leq cn^\tau$ for every $n \geq n_0$, so ω is a property of the whole family of problems, not of any one algorithm. The set is non-empty, since $\tau = 3$ belongs to it by the count above. It is bounded below, and the reason is the only thing about ω that this book proves.

Proposition 23.13.1. The exponent is at least two

$M(n) \geq n^2$ for every $n \geq 1$, and therefore $\omega \geq 2$.

Proof. Let $g_1, \dots, g_N$ be a program computing the $n \times n$ product, and fix i, j . The polynomial $\sum_k a_{ik}b_{kj}$ occurs as some g_t . It is not a constant and not an indeterminate, since it is a non-zero sum of degree-two monomials; so g_t is an arithmetic step. Distinct pairs (i, j) give distinct polynomials, because $\sum_k a_{ik}b_{kj}$ determines i and j from the indices appearing in it. Hence the n^2 pairs require n^2 distinct arithmetic steps, and the cost is at least n^2 .

Now let τ be admissible, say $M(n) \leq cn^\tau$ for all $n \geq n_0$. Then $n^2 \leq cn^\tau$, so $n^{2-\tau} \leq c$, for all $n \geq n_0$. If $\tau < 2$ the left side is unbounded, which is impossible; so $\tau \geq 2$. Every admissible τ is at least 2, and hence so is their infimum. ■

So $2 \leq \omega \leq 3$, and the question of the section is where in that interval ω lies.

Warning. The exponent is an asymptotic statement and says nothing about any particular size. $M(n) = O(n^\tau)$ hides a constant, and a program family with a small exponent may carry a constant so large that it loses to the $2n^3 - n^2$ recipe at every size anyone computes with. Section 12's $n \log n$ transform is the exception rather than the rule: there the constant is small. Below, every claim about a *size* is made with the exact operation count in hand, never from an exponent.

23.13.2 Seven products instead of eight

The classical program for $n = 2$ forms the eight products $a_{ik}b_{kj}$. Its cost is what makes the general recipe cost n^3 , because a 2×2 block form turns one product of size n into eight of size $n/2$ — and $8 = 2^3$ is exactly the recursion that reproduces n^3 . Reducing eight to seven would change the exponent.

Theorem 23.13.2. Strassen's identity

Let F be a field, let $m \geq 1$, and let $\mathbf{A}, \mathbf{B} \in M_{2m}(F)$ be partitioned into $m \times m$ blocks,

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{pmatrix}.$$

Define seven products

$$\begin{aligned} \mathbf{P}_1 &= (\mathbf{A}_{11} + \mathbf{A}_{22})(\mathbf{B}_{11} + \mathbf{B}_{22}), & \mathbf{P}_2 &= (\mathbf{A}_{21} + \mathbf{A}_{22})\mathbf{B}_{11}, \\ \mathbf{P}_3 &= \mathbf{A}_{11}(\mathbf{B}_{12} - \mathbf{B}_{22}), & \mathbf{P}_4 &= \mathbf{A}_{22}(\mathbf{B}_{21} - \mathbf{B}_{11}), \\ \mathbf{P}_5 &= (\mathbf{A}_{11} + \mathbf{A}_{12})\mathbf{B}_{22}, & \mathbf{P}_6 &= (\mathbf{A}_{21} - \mathbf{A}_{11})(\mathbf{B}_{11} + \mathbf{B}_{12}), \\ \mathbf{P}_7 &= (\mathbf{A}_{12} - \mathbf{A}_{22})(\mathbf{B}_{21} + \mathbf{B}_{22}). \end{aligned}$$

Then, with the same block partition of $\mathbf{AB}$,

$$\mathbf{AB} = \begin{pmatrix} \mathbf{P}_1 + \mathbf{P}_4 - \mathbf{P}_5 + \mathbf{P}_7 & \mathbf{P}_3 + \mathbf{P}_5 \\ \mathbf{P}_2 + \mathbf{P}_4 & \mathbf{P}_1 + \mathbf{P}_3 - \mathbf{P}_2 + \mathbf{P}_6 \end{pmatrix}.$$

Only seven products of $m \times m$ matrices occur on the right, where Theorem 7.1.1 asks for eight.

Idea. There is no derivation to reproduce: the seven products were found by Strassen, and what a proof can do is check them. What deserves watching during the check is *which* rules it uses. Every step expands a product of sums by distributivity and then cancels equal terms. No two blocks are ever interchanged, and no block is ever inverted or divided by — only the ring operations of $M_m(F)$ appear. That is the whole point, because blocks do not commute.

Proof. Expand the seven products by distributivity, keeping every factor in its place:

$$\begin{aligned} \mathbf{P}_1 &= \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{11}\mathbf{B}_{22} + \mathbf{A}_{22}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{22}, \\ \mathbf{P}_2 &= \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{11}, \\ \mathbf{P}_3 &= \mathbf{A}_{11}\mathbf{B}_{12} - \mathbf{A}_{11}\mathbf{B}_{22}, \\ \mathbf{P}_4 &= \mathbf{A}_{22}\mathbf{B}_{21} - \mathbf{A}_{22}\mathbf{B}_{11}, \\ \mathbf{P}_5 &= \mathbf{A}_{11}\mathbf{B}_{22} + \mathbf{A}_{12}\mathbf{B}_{22}, \\ \mathbf{P}_6 &= \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{21}\mathbf{B}_{12} - \mathbf{A}_{11}\mathbf{B}_{11} - \mathbf{A}_{11}\mathbf{B}_{12}, \\ \mathbf{P}_7 &= \mathbf{A}_{12}\mathbf{B}_{21} + \mathbf{A}_{12}\mathbf{B}_{22} - \mathbf{A}_{22}\mathbf{B}_{21} - \mathbf{A}_{22}\mathbf{B}_{22}. \end{aligned}$$

Take the four combinations in turn. For the $(1, 1)$ block,

$$\begin{aligned} \mathbf{P}_1 + \mathbf{P}_4 - \mathbf{P}_5 + \mathbf{P}_7 &= \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{11}\mathbf{B}_{22} + \mathbf{A}_{22}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{22} \\ &\quad + \mathbf{A}_{22}\mathbf{B}_{21} - \mathbf{A}_{22}\mathbf{B}_{11} \\ &\quad - \mathbf{A}_{11}\mathbf{B}_{22} - \mathbf{A}_{12}\mathbf{B}_{22} \\ &\quad + \mathbf{A}_{12}\mathbf{B}_{21} + \mathbf{A}_{12}\mathbf{B}_{22} - \mathbf{A}_{22}\mathbf{B}_{21} - \mathbf{A}_{22}\mathbf{B}_{22} \\ &= \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21}, \end{aligned}$$

since $\mathbf{A}_{11}\mathbf{B}_{22}$, $\mathbf{A}_{22}\mathbf{B}_{11}$, $\mathbf{A}_{22}\mathbf{B}_{22}$, $\mathbf{A}_{22}\mathbf{B}_{21}$ and $\mathbf{A}_{12}\mathbf{B}_{22}$ each occur once with each sign. For

the (1, 2) block,

$$\mathbf{P}_3 + \mathbf{P}_5 = \mathbf{A}_{11}\mathbf{B}_{12} - \mathbf{A}_{11}\mathbf{B}_{22} + \mathbf{A}_{11}\mathbf{B}_{22} + \mathbf{A}_{12}\mathbf{B}_{22} = \mathbf{A}_{11}\mathbf{B}_{12} + \mathbf{A}_{12}\mathbf{B}_{22}.$$

For the (2, 1) block,

$$\mathbf{P}_2 + \mathbf{P}_4 = \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{21} - \mathbf{A}_{22}\mathbf{B}_{11} = \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{21}.$$

For the (2, 2) block,

$$\begin{aligned} \mathbf{P}_1 + \mathbf{P}_3 - \mathbf{P}_2 + \mathbf{P}_6 &= \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{11}\mathbf{B}_{22} + \mathbf{A}_{22}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{22} \\ &\quad + \mathbf{A}_{11}\mathbf{B}_{12} - \mathbf{A}_{11}\mathbf{B}_{22} \\ &\quad - \mathbf{A}_{21}\mathbf{B}_{11} - \mathbf{A}_{22}\mathbf{B}_{11} \\ &\quad + \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{21}\mathbf{B}_{12} - \mathbf{A}_{11}\mathbf{B}_{11} - \mathbf{A}_{11}\mathbf{B}_{12} \\ &= \mathbf{A}_{21}\mathbf{B}_{12} + \mathbf{A}_{22}\mathbf{B}_{22}. \end{aligned}$$

The four results are the four blocks of $\mathbf{AB}$ given by Theorem 7.1.1. This proves the identity. ■

Two features of that proof matter more than the result. First, it used only addition, subtraction, multiplication and their laws in $M_m(F)$ — never the commutativity of a product, never an inverse, never a division. So the identity would hold with the blocks taken in any ring whatever. Second, the case $m = 1$ is the ordinary 2×2 product with seven scalar multiplications instead of eight, and the case $m > 1$ is the same statement one level up. That self-similarity is what the next theorem exploits.

Example. Seven products on integer data

Multiply $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$ by Theorem 23.13.2, with blocks of size 1, so that each $\mathbf{P}_k$ is a number. Check against the classical product.

Solution. The seven products are

$$\begin{aligned} \mathbf{P}_1 &= (1 + 4)(5 + 8) = 65, & \mathbf{P}_2 &= (3 + 4) \cdot 5 = 35, \\ \mathbf{P}_3 &= 1 \cdot (6 - 8) = -2, & \mathbf{P}_4 &= 4 \cdot (7 - 5) = 8, \\ \mathbf{P}_5 &= (1 + 2) \cdot 8 = 24, & \mathbf{P}_6 &= (3 - 1)(5 + 6) = 22, \\ \mathbf{P}_7 &= (2 - 4)(7 + 8) = -30. \end{aligned}$$

Then

$$\begin{aligned} \mathbf{P}_1 + \mathbf{P}_4 - \mathbf{P}_5 + \mathbf{P}_7 &= 65 + 8 - 24 - 30 = 19, \\ \mathbf{P}_3 + \mathbf{P}_5 &= -2 + 24 = 22, \\ \mathbf{P}_2 + \mathbf{P}_4 &= 35 + 8 = 43, \\ \mathbf{P}_1 + \mathbf{P}_3 - \mathbf{P}_2 + \mathbf{P}_6 &= 65 - 2 - 35 + 22 = 50. \end{aligned}$$

The classical product is $\begin{pmatrix} 5 + 14 & 6 + 16 \\ 15 + 28 & 18 + 32 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$, the same.

Warning. The seven products are not seven of the eight classical ones. Not one of $\mathbf{P}_1, \dots, \mathbf{P}_7$ equals a product $a_{ik}b_{kj}$; each is a product of two *combinations*. Dropping a classical product and repairing the rest is impossible, and the identity gives no license to do arithmetic in a different order in any other formula. What it gives is one algebraic fact about one bilinear map, and everything below is that fact applied to itself.

Quick check. Does [Theorem 23.13.2](#) still hold over $\mathbb{F}_2$, where $1 + 1 = 0$?

Solution. Yes. The field F was arbitrary, and the proof used only distributivity, associativity and the cancellation of a term against its negative. What changes over $\mathbb{F}_2$ is only cosmetic: every minus sign may be written as a plus sign, since $-x = x$ there.

23.13.3 The recursion, and its exact cost

Apply [Theorem 23.13.2](#) with $m \times m$ blocks, and each of the seven products is a product of two $m \times m$ matrices, to be computed by the same rule. The recursion bottoms out at 1×1 blocks, where a product is one multiplication.

Theorem 23.13.3. The cost of the recursion

Let $n = 2^p$ with $p \geq 0$. Applying [Theorem 23.13.2](#) recursively, with the 1×1 case done by a single multiplication, multiplies two matrices in $M_n(F)$ with exactly

$$T(n) = 7n^{\log_2 7} - 6n^2$$

arithmetic operations. In particular $T(n) = O(n^{\log_2 7})$ and $\omega \leq \log_2 7 = 2.8073\dots$

Idea. Two counts and one recurrence. Count the block additions in [Theorem 23.13.2](#): ten to build the operands of the seven products, eight to assemble the four blocks of the answer, eighteen in all, each of them $(n/2)^2$ scalar additions. That gives $T(n) = 7T(n/2) + \frac{9}{2}n^2$. Then solve the recurrence exactly, by induction on p rather than by a growth estimate, so that the constants are visible.

Proof. Step 1: the recurrence. Let $n = 2m$ with $m \geq 1$, and partition $\mathbf{A}$ and $\mathbf{B}$ into four $m \times m$ blocks each. Forming the operands of $\mathbf{P}_1, \dots, \mathbf{P}_7$ uses the block sums $\mathbf{A}_{11} + \mathbf{A}_{22}$, $\mathbf{B}_{11} + \mathbf{B}_{22}$, $\mathbf{A}_{21} + \mathbf{A}_{22}$, $\mathbf{B}_{12} - \mathbf{B}_{22}$, $\mathbf{B}_{21} - \mathbf{B}_{11}$, $\mathbf{A}_{11} + \mathbf{A}_{12}$, $\mathbf{A}_{21} - \mathbf{A}_{11}$, $\mathbf{B}_{11} + \mathbf{B}_{12}$, $\mathbf{A}_{12} - \mathbf{A}_{22}$ and $\mathbf{B}_{21} + \mathbf{B}_{22}$: ten of them. Assembling the answer uses three block additions for the $(1, 1)$ block, one for $(1, 2)$, one for $(2, 1)$ and three for $(2, 2)$: eight more. Each block addition is m^2 scalar additions, and the seven products cost $T(m)$ each. Hence

$$T(2m) = 7T(m) + 18m^2, \quad T(1) = 1.$$

Step 2: the solution. We show $T(2^p) = 7 \cdot 7^p - 6 \cdot 4^p$ by induction on p . For $p = 0$ the

right side is $7 - 6 = 1 = T(1)$. Suppose it holds for p . Taking $m = 2^p$, so that $m^2 = 4^p$,

$$\begin{aligned} T(2^{p+1}) &= 7(7 \cdot 7^p - 6 \cdot 4^p) + 18 \cdot 4^p \\ &= 7 \cdot 7^{p+1} - 42 \cdot 4^p + 18 \cdot 4^p \\ &= 7 \cdot 7^{p+1} - 24 \cdot 4^p = 7 \cdot 7^{p+1} - 6 \cdot 4^{p+1}, \end{aligned}$$

which is the statement for $p + 1$.

Step 3: reading it in n . With $n = 2^p$ we have $4^p = n^2$, and $7^p = 2^{p \log_2 7} = n^{\log_2 7}$. So $T(n) = 7n^{\log_2 7} - 6n^2$, which is at most $7n^{\log_2 7}$.

Finally, the recursion is a straight-line program in the sense of [Definition 23.13.1](#): it uses only additions, subtractions and multiplications, and by [Theorem 23.13.2](#) it produces every entry of $\mathbf{AB}$ as a polynomial identity. Hence $M(n) \leq 7n^{\log_2 7}$ along the powers of two. For general n , pad $\mathbf{A}$ and $\mathbf{B}$ with zero rows and columns to the next power of two, which at most doubles n ; the product of the padded matrices has $\mathbf{AB}$ in its leading block, so $M(n) \leq 7(2n)^{\log_2 7} < 49n^{\log_2 7}$ for every n . Therefore $\tau = \log_2 7$ is admissible in [Definition 23.13.2](#), and $\omega \leq \log_2 7$. ■

The number $\log_2 7 = 2.8073\dots$ is below 3, and that is the whole content: an operation that looked like an n^3 operation is not one. The mechanism is worth naming, because Section 12 used it too. There, the Fourier matrix $\mathbf{F}_n$ was *factored* into a permutation, two copies of $\mathbf{F}_{n/2}$ and a diagonal factor, and the recurrence $C(n) = 2C(n/2) + O(n)$ followed. Here the bilinear map $(\mathbf{A}, \mathbf{B}) \mapsto \mathbf{AB}$ is factored into seven products and eighteen sums, and the recurrence $T(n) = 7T(n/2) + \frac{9}{2}n^2$ follows. In both cases the saving comes from a factorization, and the exponent is decided by the *number of recursive calls* against the *size reduction*: 2 calls at half size against a linear overhead gives $n \log n$; 7 calls at half size against a quadratic overhead gives $n^{\log_2 7}$.

Remark. The recursion is not a way to make $M(2)$ smaller than 12. Its own count at $n = 2$ is $T(2) = 49 - 24 = 25$: seven multiplications but eighteen additions, against eight and four. What it does is trade one multiplication for fourteen additions at *every* level, and the trade pays only because the multiplications are what recurse.

23.13.4 What is known about the exponent, and what is not

[Theorem 23.13.3](#) gives $\omega \leq \log_2 7 < 2.8074$, and [Proposition 23.13.1](#) gives $\omega \geq 2$. Both are proved above. The rest of this subsection is **stated and not proved here**, and nothing in this book depends on any of it.

Strassen's bound was the first below 3, and it has been improved many times since. The decisive technique is the *laser method* of Coppersmith and Winograd, which brought the bound below 2.376; successive refinements of that method by several authors have since pushed the published bound below 2.372. No lower bound better than the $\omega \geq 2$ proved above is known. **Whether $\omega = 2$ is an open problem.**

Two honest caveats belong with those numbers.

- The bounds below $\log_2 7$ are obtained by analyzing families of recursive constructions, not by exhibiting seven-product identities; the algorithms they yield carry constants so large that at every size that arises in practice they are slower than the $2n^3 - n^2$ recipe. This is a known feature of the constructions, and it is recorded here, not proved.
- ω may depend on the field F , and the definition above was made field by field for that reason. The bounds just quoted, and Strassen's, hold over every field.

Remark. The name **bilinear complexity** belongs to a related quantity: the least number of *multiplications* (as opposed to all arithmetic operations) needed to compute a bilinear map, which for the 2×2 product is exactly 7. The lower bound 7 is a theorem of Winograd and is not proved here; [Theorem 23.13.2](#) is the upper half of it. The exponent ω is the same whether it is defined through [Definition 23.13.1](#) or through multiplications alone, which is also not proved here and is not used.

23.13.5 Accuracy: a norm bound, but not an entrywise one

Everything so far was exact algebra. In the floating-point model of Definition 23.1.1 the two algorithms are not interchangeable, and the difference is sharper than “one is less accurate”.

Start with the classical recipe, which inherits Section 1's analysis directly. Write $|\mathbf{A}|$ for the entrywise absolute value ($|a_{ij}|$), as in Chapter 18, and $\hat{\mathbf{C}}$ for the computed product. Everything below is real, because Definition 23.1.1 is.

Proposition 23.13.4. The classical product has a small entrywise error

Let $\mathbf{A}, \mathbf{B} \in M_n(\mathbb{R})$ have representable entries, let $nu < 1$, and let $\hat{\mathbf{C}}$ be the product computed by Definition 0.9.9, each entry as one inner product taken in the natural order. Then

$$|\hat{\mathbf{C}} - \mathbf{AB}| \leq \gamma_n |\mathbf{A}| |\mathbf{B}|$$

entrywise.

Proof. Fix i and j , and let $\mathbf{x}$ be row i of $\mathbf{A}$ and $\mathbf{y}$ column j of $\mathbf{B}$, both with representable entries. By Definition 0.9.9, $(\mathbf{AB})_{ij} = \mathbf{x}^\top \mathbf{y}$, and $\hat{c}_{ij}$ is exactly the quantity s_n of Theorem 23.1.2. That theorem, applicable since $nu < 1$, gives a $\Delta \mathbf{x}$ with $\hat{c}_{ij} = (\mathbf{x} + \Delta \mathbf{x})^\top \mathbf{y}$ and $|\Delta x_k| \leq \gamma_n |x_k|$ for every k . Hence

$$|\hat{c}_{ij} - (\mathbf{AB})_{ij}| = \left| \sum_k \Delta x_k y_k \right| \leq \sum_k |\Delta x_k| |y_k| \leq \gamma_n \sum_k |a_{ik}| |b_{kj}|,$$

where the first inequality is the triangle inequality and the second substitutes the bound on $\Delta \mathbf{x}$ together with $|x_k| = |a_{ik}|$. The right side is $\gamma_n (|\mathbf{A}| |\mathbf{B}|)_{ij}$. Since i and j were arbitrary, this proves the bound. ■

The bound is **entrywise**, and that is its strength: the error in the (i, j) entry is measured

against the size of the data that produced *that* entry. If $(|\mathbf{A}||\mathbf{B}|)_{ij}$ is tiny, the error there is tiny too. In particular, if $\mathbf{A}$ and $\mathbf{B}$ have non-negative entries, then $(|\mathbf{A}||\mathbf{B}|)_{ij} = (\mathbf{AB})_{ij}$ and every entry is computed with small *relative* error.

Strassen's algorithm satisfies a weaker bound, in which the whole of $\mathbf{A}$ and the whole of $\mathbf{B}$ appear on the right. Here is the base case, proved.

Proposition 23.13.5. Strassen's two-by-two product satisfies a norm bound

Let $\mathbf{A}, \mathbf{B} \in M_2(\mathbb{R})$ have representable entries, put $\alpha = \max_{i,j} |a_{ij}|$ and $\beta = \max_{i,j} |b_{ij}|$, and let $\hat{\mathbf{C}}$ be the result of evaluating [Theorem 23.13.2](#) with $m = 1$, each $\mathbf{P}_k$ as written and each of the four combinations left to right. If $6u < 1$, then

$$\max_{i,j} |\hat{c}_{ij} - (\mathbf{AB})_{ij}| \leq 12\gamma_6 \alpha\beta.$$

Idea. Each $\mathbf{P}_k$ is built by at most three operations, so the computed $\hat{\mathbf{P}}_k$ carries at most three factors $1 + \delta$. Each output block is a sum of at most four of them, formed by at most three further additions, so each summand carries at most three more. Six factors in all, which is γ_6 by Lemma 23.1.1. The only other ingredient is a crude size bound on each $\mathbf{P}_k$.

Proof. Sizes. Each factor of $\mathbf{P}_k$ is either a single entry or a sum of two, so $|\mathbf{P}_1|, |\mathbf{P}_6|, |\mathbf{P}_7| \leq (2\alpha)(2\beta) = 4\alpha\beta$, while $\mathbf{P}_2, \mathbf{P}_3, \mathbf{P}_4, \mathbf{P}_5$ each have a single entry as one factor and so satisfy $|\mathbf{P}_k| \leq 2\alpha\beta$.

Rounding. Fix k . Forming $\mathbf{P}_k$ takes at most one addition on each side and then one multiplication: at most three operations, each contributing a factor $1 + \delta$ with $|\delta| \leq u$ by Definition 23.1.1. So $\hat{\mathbf{P}}_k$ is $\mathbf{P}_k$ times a product of at most three such factors.

Assembly. Take $\hat{c}_{11}$, computed as $((\hat{\mathbf{P}}_1 + \hat{\mathbf{P}}_4) - \hat{\mathbf{P}}_5) + \hat{\mathbf{P}}_7$. Each of the three additions contributes one factor $1 + \delta$ to every summand already accumulated, so $\hat{\mathbf{P}}_1$ and $\hat{\mathbf{P}}_4$ pick up three such factors, $\hat{\mathbf{P}}_5$ two and $\hat{\mathbf{P}}_7$ one. Combining with the previous paragraph, each $\mathbf{P}_m$ is multiplied by a product of at most six factors $1 + \delta$, which by Lemma 23.1.1 is $1 + \psi_m$ with $|\psi_m| \leq \gamma_6$, legitimate because $6u < 1$. Hence

$$\hat{c}_{11} = \sum_{m \in \{1,4,5,7\}} \pm \mathbf{P}_m (1 + \psi_m),$$

with the signs of [Theorem 23.13.2](#). Subtracting $c_{11} = \mathbf{P}_1 + \mathbf{P}_4 - \mathbf{P}_5 + \mathbf{P}_7$ and using the triangle inequality,

$$|\hat{c}_{11} - c_{11}| \leq \gamma_6 \sum_{m \in \{1,4,5,7\}} |\mathbf{P}_m| \leq \gamma_6 (4 + 2 + 2 + 4)\alpha\beta = 12\gamma_6 \alpha\beta.$$

The same argument gives $|\hat{c}_{22} - c_{22}| \leq \gamma_6 (4 + 2 + 2 + 4)\alpha\beta = 12\gamma_6 \alpha\beta$ for the sum $\mathbf{P}_1 + \mathbf{P}_3 - \mathbf{P}_2 + \mathbf{P}_6$. The blocks $\hat{c}_{12} = \hat{\mathbf{P}}_3 + \hat{\mathbf{P}}_5$ and $\hat{c}_{21} = \hat{\mathbf{P}}_2 + \hat{\mathbf{P}}_4$ use one addition, so four factors and $\gamma_4 \leq \gamma_6$ suffice, and $\sum_m |\mathbf{P}_m| \leq 4\alpha\beta$ there. The largest of the four bounds is $12\gamma_6 \alpha\beta$, which proves the proposition. ■

A bound of this shape survives the recursion, with a constant that grows with n : for $n = 2^p$ there is a constant $c(n)$, polynomial in n , with

$$\max_{i,j} |\hat{c}_{ij} - (\mathbf{AB})_{ij}| \leq c(n) u \max_{i,j} |a_{ij}| \max_{i,j} |b_{ij}| + O(u^2).$$

This is not proved here. A bound of this form was first established by Brent, and the constant has been improved since; the induction is longer than this section, and nothing below uses it. What *is* proved below is that no improvement of it can be entrywise.

Theorem 23.13.6. Strassen's algorithm has no useful entrywise error bound

Work in a model as in Definition 23.1.1, and suppose the set $\mathcal{R}$ of representable numbers contains a number ε with

$$\varepsilon > 0, \quad 1 + \varepsilon \in \mathcal{R}, \quad \varepsilon^2 \in \mathcal{R}, \quad \text{fl}((1 + \varepsilon)^2) = 1 + 2\varepsilon.$$

Put $\mathbf{A} = \mathbf{B} = \text{diag}(1, \varepsilon)$. Then the $\hat{\mathbf{C}}$ of Proposition 23.13.5 has $\hat{c}_{22} = 0$, while $(\mathbf{AB})_{22} = \varepsilon^2$. Consequently, any constant c for which

$$|\hat{\mathbf{C}} - \mathbf{AB}| \leq cu |\mathbf{A}| |\mathbf{B}|$$

holds entrywise for this $\mathbf{A}$ and $\mathbf{B}$ must satisfy $c \geq 1/u$.

Idea. Choose the data so that the $(2, 2)$ entry of the product is as small as the model can see, while the quantity $\mathbf{P}_1$ from which it is assembled has size 1. The term ε^2 is then the last bit of $(1 + \varepsilon)^2$, and the fourth hypothesis says the rounding throws exactly that bit away. Nothing later can put it back, because the remaining steps are exact.

Proof. Write $a_{11} = b_{11} = 1$, $a_{22} = b_{22} = \varepsilon$ and $a_{12} = a_{21} = b_{12} = b_{21} = 0$; all are in $\mathcal{R}$, since $0, 1, \varepsilon \in \mathcal{R}$. Note first that fl fixes every element of $\mathcal{R}$, because a nearest element of $\mathcal{R}$ to a member of $\mathcal{R}$ is that member. By Definition 0.9.9, $(\mathbf{AB})_{22} = a_{21}b_{12} + a_{22}b_{22} = \varepsilon^2$.

Compute the four quantities that Theorem 23.13.2 combines into the $(2, 2)$ entry. The operand sums are $\text{fl}(a_{11} + a_{22}) = \text{fl}(1 + \varepsilon) = 1 + \varepsilon$ and likewise for $\mathbf{B}$, both exact by hypothesis; so

$$\hat{\mathbf{P}}_1 = \text{fl}((1 + \varepsilon)(1 + \varepsilon)) = \text{fl}((1 + \varepsilon)^2) = 1 + 2\varepsilon,$$

by the fourth hypothesis. Next $\hat{\mathbf{P}}_2 = \text{fl}(\text{fl}(0 + \varepsilon) \cdot 1) = \varepsilon$, and $\hat{\mathbf{P}}_3 = \text{fl}(1 \cdot \text{fl}(0 - \varepsilon)) = -\varepsilon$, using $-\mathcal{R} = \mathcal{R}$; and $\hat{\mathbf{P}}_6 = \text{fl}(\text{fl}(0 - 1) \cdot \text{fl}(1 + 0)) = \text{fl}(-1) = -1$. Each of these values lies in $\mathcal{R}$, so each rounding is the identity.

Assemble $\mathbf{P}_1 + \mathbf{P}_3 - \mathbf{P}_2 + \mathbf{P}_6$ left to right. In order,

$$\text{fl}((1 + 2\varepsilon) + (-\varepsilon)) = 1 + \varepsilon, \quad \text{fl}((1 + \varepsilon) - \varepsilon) = 1, \quad \text{fl}(1 + (-1)) = 0,$$

every intermediate value being in $\mathcal{R}$ and so fixed by fl . Hence $\hat{c}_{22} = 0$.

Finally $|\mathbf{A}| |\mathbf{B}| = \text{diag}(1, \varepsilon^2)$, so an entrywise bound at the $(2, 2)$ position reads $\varepsilon^2 = |0 - \varepsilon^2| \leq cu\varepsilon^2$. Dividing by $\varepsilon^2 > 0$ gives $1 \leq cu$, that is $c \geq 1/u$. This proves the theorem. ■

The constant $1/u$ is about 10^{16} in the arithmetic in common use, so the conclusion is that no entrywise bound worth stating exists: [Proposition 23.13.4](#) holds with $c = \gamma_2/u$, which is close to 2, and for Strassen's algorithm the same inequality at $n = 2$ needs a constant larger than the reciprocal of the unit roundoff. The hypotheses of the theorem are not exotic.

They are not vacuous either, and the fourth one fixes the scale of ε . Since $1 + \varepsilon$ is representable, Definition 23.1.1 writes $\text{fl}((1 + \varepsilon)^2) = (1 + \varepsilon)^2(1 + \delta)$ with $|\delta| \leq u$, and the fourth hypothesis says the left side is $1 + 2\varepsilon$, which forces

$$\delta = -\frac{\varepsilon^2}{(1 + \varepsilon)^2}, \quad \text{hence} \quad \varepsilon^2 \leq u(1 + \varepsilon)^2.$$

So ε is at most about $\sqrt{u}$: the theorem asks for a quantity small enough that its square falls off the end of the significand, and for nothing else. The next example produces one.

Example. A small entry lost entirely

Work in binary arithmetic with 53-bit significands, rounding to nearest, so that $u = 2^{-53}$. Verify the hypotheses of [Theorem 23.13.6](#) for $\varepsilon = 2^{-27}$, and say what each algorithm returns for the $(2, 2)$ entry of the product of $\mathbf{A} = \mathbf{B} = \text{diag}(1, 2^{-27})$.

Solution. The hypotheses. A number is representable when its significand fits in 53 bits. Now $\varepsilon = 2^{-27}$ is a power of two, so $\varepsilon \in \mathcal{R}$ and $\varepsilon^2 = 2^{-54} \in \mathcal{R}$. The number $1 + 2^{-27}$ needs bits 0 through 27, that is 28 of them, so it is representable. Finally

$$(1 + 2^{-27})^2 = 1 + 2^{-26} + 2^{-54},$$

which needs 55 bits and is not. In $[1, 2)$ the representable numbers are spaced 2^{-52} apart; our value lies 2^{-54} above $1 + 2^{-26}$ and $3 \cdot 2^{-54}$ below the next one up, so rounding to nearest gives $\text{fl}((1 + 2^{-27})^2) = 1 + 2^{-26} = 1 + 2\varepsilon$. All four hypotheses hold.

The two answers. By [Theorem 23.13.6](#), Strassen's algorithm returns $\hat{c}_{22} = 0$, while the true value is 2^{-54} : the relative error is 1, not a loss of digits but a loss of all of them. The classical recipe computes $\text{fl}(0 \cdot 0) = 0$ and then $\text{fl}(0 + \text{fl}(2^{-27} \cdot 2^{-27})) = \text{fl}(2^{-54}) = 2^{-54}$, the exact answer.

The other three entries of the Strassen product also come out exactly right here, and the error 2^{-54} is invisible when measured against $\max|a_{ij}|\max|b_{ij}| = 1$, which is what [Proposition 23.13.5](#) measures it against. That is the whole contrast in one example.

Warning. “No entrywise bound” is not “unstable”. [Proposition 23.13.5](#) is a genuine bound, and for a matrix whose entries are all of comparable size it says as much as [Proposition 23.13.4](#) does. The two part company only when the entries of $\mathbf{AB}$ span many orders of magnitude, and then the classical algorithm delivers something Strassen's does not: small relative error in the small entries. Backward stability in the sense of Definition 23.1.3 is likewise not in question — the failure above is a failure of *entrywise* accuracy, exactly as Chapter 15 §08 warned that conditioning and stability answer different questions.

23.13.6 Whether to use it

The operation count and the error analysis pull in opposite directions, and the honest summary is a list of exact numbers rather than a verdict. Suppose the recursion is stopped at a base size n_0 , below which the classical recipe is used, and let $n = n_0 2^p$. The same induction as in [Theorem 23.13.3](#), started from $T(n_0) = 2n_0^3 - n_0^2$, gives

$$T(n_0 2^p) = 7^p(2n_0^3 - n_0^2) + 6n_0^2(7^p - 4^p).$$

Three readings of this formula. With $n_0 = 1$ the recursion runs to scalars, and it is *slower* than the classical recipe until n exceeds about 654; at $n = 512$ it costs 280 902 385 operations against 268 173 312, and at $n = 1024$ it costs 1 971 035 287 against 2 146 435 072. With a classical base case the picture changes completely: a *single* level of the recursion, $p = 1$, costs $14n_0^3 + 11n_0^2$ against the classical $16n_0^3 - 4n_0^2$ at size $2n_0$, and the first wins as soon as $2n_0^3 > 15n_0^2$, that is from $n_0 = 8$, so from $n = 16$. And at $n = 1024$ with $n_0 = 64$ the count is 1 301 696 512, about 0.61 of the classical one.

So on operation count alone the trade is favorable at sizes that are not large. Three things keep the classical algorithm the one in ordinary use at small and moderate sizes, and none of them is visible in an operation count.

- **Additions are not free in the way the count assumes.** The seven products must be stored, and the eighteen block sums read and write whole blocks that the classical recipe never touches. An operation count charges 1 for an addition and 1 for a multiplication and charges nothing for moving a number; on a real machine the movement is often the expensive part, and the recursion moves much more data than it saves in arithmetic.
- **The error bound is weaker, and it is weaker in a way that matters exactly when it is not expected** — see [Example \(A small entry lost entirely\)](#).
- **The sizes have to cooperate.** The recursion wants even dimensions at every level; an odd size must be padded or handled specially, and the bookkeeping erodes the saving at precisely the moderate sizes where it is thinnest.

The conclusion is not that [Theorem 23.13.2](#) is a curiosity. It is used, at large sizes, with a classical base case and with the accuracy caveat understood. What it is not is a replacement for a definition: **AB** still means what Definition 0.9.9 says it means, and every theorem in this book about products is untouched. A fast algorithm changes the cost of an operation, never the operation.

23.13.7 Summary and transfer

Chapter 20 closed Part V with a complaint and a promise: “every proof here is an existence or a comparison, and not one of them is an algorithm. That is the subject of Chapter 23.” Thirteen sections later, the promise is paid, and what “compute” turned out to mean can be said in one sentence: a computed answer is the exact answer to a nearby question, and how near depends on the algorithm, while how much that nearness costs depends on the problem.

The chapter’s one new assumption was the floating-point model of Definition 23.1.1, a description of a machine stated as a definition. Everything else was proved from it and from the linear algebra of Chapters 0 to 20.

- **What a computed answer means.** Section 1 stated the model, proved Lemma 23.1.1 and Theorem 23.1.2, and gave the words the rest of the chapter needs: Definition 23.1.3, Definition 23.1.2 and the rule Theorem 23.1.3, that the forward error is at most the condition number times the backward error.

- **Solving a system.** Section 2 read elimination as a sequence of Schur complements (Theorem 23.2.1), counted it (Proposition 23.2.3), and proved backward error analyses in the cases it could reach: Theorem 23.2.5 outright, and Theorem 23.2.6 for the ordering it displays, the same bound for Chapter 2’s ordering being stated with credit. With it came the growth factor Definition 23.2.1, the bound Theorem 23.2.8 — attained, by Example (A matrix that really grows) — and the pivot-free safety of Cholesky (Theorem 23.2.9).
- **Least squares.** Section 3 proved that the sensitivity of the least-squares solution involves the residual as well as $\kappa_2(\mathbf{A})$ (Theorem 23.3.1), that the normal equations lose twice the digits (Theorem 23.3.2), and that a reflection does not (Theorem 23.3.3) — that last for one reflection applied to a matrix, the accumulation over the whole factorization being stated with credit.
- **Eigenvalues.** Section 4 built the power method (Theorem 23.4.1), inverse iteration (Theorem 23.4.3) and Rayleigh quotient iteration (Theorem 23.4.4); Section 5 identified the QR algorithm with orthogonal iteration (Theorem 23.5.2), proved its convergence (Theorem 23.5.3), and explained the Hessenberg reduction (Proposition 23.5.4) and the shifts.
- **Iterating in a subspace.** Section 6 built the Krylov space (Definition 23.6.1), the Arnoldi process (Theorem 23.6.2) and its Hermitian three-term form (Theorem 23.6.3), and showed that the Ritz values (Definition 23.6.2) interlace (Theorem 23.6.4) and carry Chapter 19’s residual bounds. Section 7 minimized over the same space: Theorem 23.7.4, the Chebyshev bound Lemma 23.7.6, and the square root in Theorem 23.7.7.
- **One piece of pure mathematics.** Section 8 showed that an algorithm and a differential equation can be the same object: the Lax pair Definition 23.8.1, the isospectral theorem Theorem 23.8.1, and the Toda flow’s relation to the QR algorithm (Theorem 23.8.5).
- **Applications.** Section 9 turned a mass-and-spring system into a generalized eigenproblem (Theorem 23.9.2); Section 10 proved that the best-fitting affine subspace is spanned by the leading singular vectors of the centered data matrix (Theorem 23.10.2) and turned “variance explained” into a theorem (Theorem 23.10.4); Section 11 built the graph Laplacian (Definition 23.11.1), identified its kernel (Theorem 23.11.2), introduced the Fiedler value (Definition 23.11.2) with the minimum it solves (Proposition 23.11.4) and the partition the signs of a Fiedler vector give, and proved the matrix-tree theorem (Theorem 23.11.8).
- **Speed.** Section 12 factored the Fourier matrix (Theorem 23.12.1) and solved the resulting recurrence (Theorem 23.12.2); this section factored the two-by-two product (Theorem 23.13.2), solved the resulting recurrence (Theorem 23.13.3), proved the one lower bound on ω this book has (Proposition 23.13.1), and showed what the speed costs (Theorem 23.13.6).

Four moves carried the chapter.

- **Conditioning belongs to the problem, stability to the algorithm, and the two multiply.** Chapter 15 §08 stated the distinction and could not use it; Theorem 23.1.3 makes it a theorem, and every accuracy claim in the chapter is an instance. *Transfer:* when an answer is wrong, ask first whether any method could have done better. If the condition number says no, do not change the algorithm — change the problem.
- **Measure an answer by a change in the data that makes it exact.** A forward error is usually unknowable and a backward error is usually computable, so prove the backward statement and let Theorem 23.1.3 supply the rest (§§01, 02, 03). *Transfer:* when a quantity you want to bound is out of reach, look for a perturbation of the input that the computed output answers exactly; bounding that is often elementary.
- **Every eigenvalue algorithm iterates, because no finite one can exist.** Chapter 10 §08 gave the reason — a finite procedure using arithmetic and radicals would solve the general quintic — and Sections 4 to 7 are what is left once that door is shut: a sequence of subspaces, each cheap to extend, each carrying a certified error bar from Chapter 19.

Transfer: when exact solution is impossible in principle, the design question becomes what to iterate *in*; the Krylov space is the answer whenever the only affordable operation is one matrix–vector product.

- **A fast algorithm is a factorization.** The Fourier matrix factors into sparse pieces and the two-by-two product factors into seven multiplications; in both cases the cost follows by solving a recurrence exactly, and in both cases the underlying object is unchanged (§§12, 13). *Transfer:* to make an operation fast, do not look for a shortcut in the arithmetic. Look for a factorization of the operation itself, and count the recursive calls against the size reduction.

The chapter paid the following promises, in the words in which they were made.

The promise	Made in	Paid by
“whether a given method keeps that promise is stability, and it is Chapter 23’s subject”	Chapter 15 §08	§01
“Chapter 23 draws the distinction properly, defines backward stability, and proves which of the algorithms in this book have it”	Chapter 15 §08	§01, §02, §03
“An error analysis of this kind, which measures an answer by a change in the data that makes it exact, is a backward error analysis . Chapter 23 builds on it.”	Chapter 19 §04	§01
“Chapter 23 makes this precise” (the ε -pivot failure)	Chapter 2 §06	§02
“Chapter 23 reads Gaussian elimination itself as a sequence of Schur complements of 1×1 pivots”	Chapter 7 §03	§02
“Cholesky does about half the arithmetic of a general LU factorization ... it needs no pivoting”	Chapter 12 §02	§02
“Three things are <i>not</i> proved here, and all three belong to Chapter 23” (the second of them being that nothing in that chapter is about arithmetic)	Chapter 15 §08	§01, §02, §03
“The conditioning of the least-squares problem itself is not $\kappa_2(\mathbf{A})$... a term in $\kappa_2(\mathbf{A})^2$ appears even for the QR route”	Chapter 15 §08	§03

The promise	Made in	Paid by
“Chapter 23 makes ‘sensitivity’ precise and proves the comparison”	Chapter 10 §08, quoted again in Chapter 15 §08	§03
“a stable implementation of the QR route loses about $\log_{10} \kappa_2(\mathbf{A})$ decimal digits while forming and solving the normal equations loses about $2\log_{10} \kappa_2(\mathbf{A})$ — is still Chapter 23’s to prove”	Chapter 15 §08	§03
“Chapter 23 quantifies the loss” (the Householder sign choice)	Chapter 10 §08	§03
“Chapter 23 explains why it is the stable one” (reflections against classical Gram–Schmidt)	Chapter 10 §02	§03
“Chapter 23 builds an iteration on it” (the Rayleigh quotient)	Chapter 16 §01 and Chapter 19 §06	§04
“the iteration $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$ is the power method, and Chapter 23 studies how fast it converges”	Chapter 18 §04	§04
“Chapter 23 studies it for general matrices, where the rate is governed by the ratio of the two largest eigenvalue moduli”	Chapter 18 §08	§04
“the power method, the algorithm that computes a Perron vector at the size of the web, belongs to Chapter 23”	Chapter 18 §09	§04
“Chapter 23 develops that iteration and explains why every implementation starts with the reduction proved here”	Chapter 10 §08	§05
“algorithms that compute eigenvalues, is the subject of Chapter 23”	Chapter 19 §11	§04, §05, §06
“Those steps need a probabilistic model, a reason to prefer squared error, and an argument that the sample answer says something about the population. Chapter 23 supplies them”	Chapter 12 §10	§10, for the first two; the third is refused, below

The promise	Made in	Paid by
“the graph Laplacian, returns in Chapter 23”	Chapter 2 §07	§11
the matrix-tree theorem, “which Chapter 23 completes”	Chapter 6 §07	§11
“Chapter 23 develops it, together with the cost accounting that makes ‘quickly’ precise” (the fast Fourier transform)	Chapter 11 §09	§12

Some things the chapter did not do, and the chapter’s index collects them in one list. Section 2 proved the backward error of a factorization for the ordering it displays and stated with credit that Chapter 2’s ordering satisfies the same bound; it also showed that the growth bound for partial pivoting is far too large to prove stability, and said so rather than tidying it. Section 3 proved the backward error of a single reflection and stated with credit both the accumulation over a whole factorization and the rates at which classical and modified Gram–Schmidt lose orthogonality. Section 4 stated, and did not prove, that Rayleigh quotient iteration converges at a quadratic rate, not a cubic one, for a general diagonalizable matrix. Section 5 quoted the theorem of Abel and Ruffini, which is why every eigenvalue method here iterates, and proved the first column of the convergence theorem of the QR algorithm while stating the general form with credit. Section 6 stated the classical convergence estimate for the extreme Ritz values and proved the clean special case alone, and described the loss of orthogonality in Lanczos without analyzing it. Section 8 derived the continuous Toda statement only after granting that $t \mapsto \mathbf{Q}(t)$ is differentiable, which is analysis this book has not set up; its discrete half, Theorem 23.8.5, rests on nothing unproved. Section 10 proved every deterministic statement about principal components and said plainly that the inference from a sample to a larger population is statistics, which this book does not develop; Chapter 12 §10 promised that inference, and promised as well an account of how a singular value decomposition is actually computed for a large matrix, which no section here gives — §10 says which route to avoid and why, and points at the orthogonal reductions of §§03 and 05, and there it stops. Both over-promises are recorded rather than hidden. In this section, the recursive form of Strassen’s error bound, Winograd’s lower bound of seven multiplications, the equality of the two definitions of ω , everything known about ω below $\log_2 7$, and Laderman’s twenty-three-product identity were stated with credit and not proved.

Nothing in this book is proved from any of them. Each is quoted exactly where a reader would otherwise expect a proof, and every theorem in the chapter either proves its own case or carries what it needs as an explicit hypothesis; since all of them enter the book here, no earlier chapter can rest on them either. The one place a quoted result is used at all is [Exercise \(C1: Would twenty-three products for three-by-three beat Strassen?\)](#) (b), where Laderman’s identity is handed to the reader as a hypothesis and the deduction from it is the reader’s own. Chapter 20’s quoted fact (A7) was not used anywhere in this chapter, and no analysis beyond Chapter 15’s (A1)–(A6) was imported.

The book’s account of computation ends here. Twenty chapters built objects and proved things about them; this one asked what it costs to produce those objects from data of finite accuracy, and found that the answer splits cleanly in two. How much accuracy the data deserve is a question about the problem, answered by a condition number. How much of that the algorithm delivers is a question about the algorithm, answered by a backward error. Everything in this

chapter — the pivoting, the reflections, the shifts, the Krylov spaces, the seven products — is an attempt to keep the second small when the first is out of our hands.

23.13.8 Exercises

23.13.8.1 A. Check your understanding

Exercise. A1

- State how many arithmetic operations Definition 0.9.9 uses for the product of two $n \times n$ matrices, separating multiplications from additions.
- Define the exponent ω , and state what is proved about it in this section.
- True or false: Theorem 23.13.2 shows that $M(2) \leq 7$. Justify your answer.
- Which properties of $m \times m$ matrices does the proof of Theorem 23.13.2 use? Say what would go wrong if the identity had needed two blocks to commute.
- State the difference between the error bound of Proposition 23.13.4 and that of Proposition 23.13.5, and say for which matrices the difference matters.

Solution.

- n^3 multiplications and $n^2(n-1)$ additions, $2n^3 - n^2$ operations in all: each of the n^2 entries is a sum of n products, needing n multiplications and $n-1$ additions.
- $\omega = \inf\{\tau : M(n) = O(n^\tau)\}$, with $M(n)$ the arithmetic cost of Definition 23.13.1. Proved here: $\omega \geq 2$ (Proposition 23.13.1) and $\omega \leq \log_2 7$ (Theorem 23.13.3). Everything else quoted about ω is stated with credit and not proved.
- False, and for two reasons. The cost of Definition 23.13.1 counts additions as well as multiplications, and Theorem 23.13.2 uses 18 of them, so its cost at $n = 2$ is 25, not 7; indeed $M(2) \leq 12$ by the classical program, which is better. The number 7 counts multiplications only.
- Only the associativity and distributivity of matrix addition and multiplication and the existence of additive inverses, together with Theorem 7.1.1, which says that the blocks of $\mathbf{AB}$ are the eight products combined as for a 2×2 matrix. No block is ever inverted and no two are ever interchanged. If some cancellation had required, say, $\mathbf{A}_{11}\mathbf{B}_{22} = \mathbf{B}_{22}\mathbf{A}_{11}$, it would fail for general blocks, which do not commute, and the identity would be true for numbers and false for matrices — so the recursion of Theorem 23.13.3 would be unavailable, which is the only reason the identity is of any interest.
- Proposition 23.13.4 bounds the (i, j) error by $\gamma_n(|\mathbf{A}||\mathbf{B}|)_{ij}$, a quantity attached to that entry; Proposition 23.13.5 bounds every error by a multiple of $\max|a_{ij}|\max|b_{ij}|$, a quantity attached to the whole matrices. They agree in strength when all entries of $\mathbf{AB}$ are of comparable size, and differ when some entry of the product is far smaller than others, as in Example (A small entry lost entirely).

23.13.8.2 B. Practice

Exercise. B1: Seven products by hand

Let $\mathbf{A} = \begin{pmatrix} 2 & -1 \\ 0 & 3 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 1 & 4 \\ -2 & 1 \end{pmatrix}$. Compute $\mathbf{P}_1, \dots, \mathbf{P}_7$ of [Theorem 23.13.2](#) with blocks of size 1, assemble the four entries, and check the answer against Example (A product of square matrices). Hence state how many multiplications and how many additions you performed.

Solution. With $a_{11} = 2, a_{12} = -1, a_{21} = 0, a_{22} = 3$ and $b_{11} = 1, b_{12} = 4, b_{21} = -2, b_{22} = 1$,

$$\begin{aligned} \mathbf{P}_1 &= (2 + 3)(1 + 1) = 10, & \mathbf{P}_2 &= (0 + 3) \cdot 1 = 3, \\ \mathbf{P}_3 &= 2 \cdot (4 - 1) = 6, & \mathbf{P}_4 &= 3 \cdot (-2 - 1) = -9, \\ \mathbf{P}_5 &= (2 - 1) \cdot 1 = 1, & \mathbf{P}_6 &= (0 - 2)(1 + 4) = -10, \\ \mathbf{P}_7 &= (-1 - 3)(-2 + 1) = 4. \end{aligned}$$

Then

$$\begin{aligned} c_{11} &= 10 - 9 - 1 + 4 = 4, & c_{12} &= 6 + 1 = 7, \\ c_{21} &= 3 - 9 = -6, & c_{22} &= 10 + 6 - 3 - 10 = 3. \end{aligned}$$

So $\mathbf{AB} = \begin{pmatrix} 4 & 7 \\ -6 & 3 \end{pmatrix}$, which is the product computed in Example (A product of square matrices).

The count: seven multiplications, and ten additions to form the operands together with eight to assemble the answer, so eighteen additions, twenty-five operations in all. The classical route uses eight multiplications and four additions, twelve operations. At this size the exchange is a loss.

Exercise. B2: One block, one level

Let

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 2 \\ 0 & 2 & 1 & 1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 2 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{pmatrix},$$

partitioned into 2×2 blocks. Compute $\mathbf{P}_3$ and $\mathbf{P}_5$ of [Theorem 23.13.2](#), hence the $(1, 2)$ block of $\mathbf{AB}$, and verify it against the block formula $\mathbf{A}_{11}\mathbf{B}_{12} + \mathbf{A}_{12}\mathbf{B}_{22}$.

Solution. The blocks are

$$\mathbf{A}_{11} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{A}_{12} = \begin{pmatrix} 2 & 1 \\ 1 & 0 \end{pmatrix}, \quad \mathbf{B}_{12} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \mathbf{B}_{22} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

Then $\mathbf{B}_{12} - \mathbf{B}_{22} = \begin{pmatrix} -1 & 0 \\ 1 & -1 \end{pmatrix}$ and $\mathbf{A}_{11} + \mathbf{A}_{12} = \begin{pmatrix} 3 & 1 \\ 1 & 1 \end{pmatrix}$, so

$$\mathbf{P}_3 = \mathbf{A}_{11}(\mathbf{B}_{12} - \mathbf{B}_{22}) = \begin{pmatrix} -1 & 0 \\ 1 & -1 \end{pmatrix}, \quad \mathbf{P}_5 = \begin{pmatrix} 3 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}.$$

Hence

$$\mathbf{P}_3 + \mathbf{P}_5 = \begin{pmatrix} 2 & 4 \\ 2 & 1 \end{pmatrix}.$$

Directly, $\mathbf{A}_{11}\mathbf{B}_{12} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and $\mathbf{A}_{12}\mathbf{B}_{22} = \begin{pmatrix} 2 & 3 \\ 1 & 1 \end{pmatrix}$, whose sum is $\begin{pmatrix} 2 & 4 \\ 2 & 1 \end{pmatrix}$. The two agree, and by Theorem 7.1.1 this is rows 1, 2 and columns 3, 4 of $\mathbf{AB}$.

Exercise. B3: Counting the recursion

Let $n = 2^p$.

- How many multiplications does the recursion of Theorem 23.13.3 perform, as a function of p ? How many does the classical recipe perform?
- Using $T(n) = 7n^{\log_2 7} - 6n^2$, compute $T(n)$ and $2n^3 - n^2$ for $n = 8$ and $n = 64$.
- Hence say, for these two sizes, which recipe uses fewer operations.

Solution.

- Each level replaces one product of size $2m$ by seven of size m , and a product of size 1 is one multiplication, so the recursion performs $7^p = n^{\log_2 7}$ multiplications. The classical recipe performs $n^3 = 8^p$.
- For $n = 8$, that is $p = 3$: $T(8) = 7 \cdot 343 - 6 \cdot 64 = 2401 - 384 = 2017$, while $2 \cdot 512 - 64 = 960$. For $n = 64$, that is $p = 6$: $T(64) = 7 \cdot 117\,649 - 6 \cdot 4096 = 823\,543 - 24\,576 = 798\,967$, while $2 \cdot 262\,144 - 4096 = 520\,192$.
- At both sizes the classical recipe wins, by a factor of about 2.1 at $n = 8$ and about 1.5 at $n = 64$. The ratio is shrinking, and it crosses 1 between $n = 512$ and $n = 1024$; recursing all the way to scalars is a losing strategy at any size a person would compute by hand.

23.13.8.3 C. Going deeper

Exercise. C1: Would twenty-three products for three-by-three beat Strassen?

Suppose a ring identity multiplies two 3×3 matrices with $q > 9$ multiplications of combinations of entries, together with some fixed number of additions, and is used recursively on 3×3 block matrices.

- Show that the recursion performs q^p multiplications on matrices of size $n = 3^p$, and deduce that it gives $\omega \leq \log_3 q$.
- You are given that such an identity exists with $q = 23$; this is a result of Laderman, which is not proved here. Determine whether recursing on it beats Theorem 23.13.3. Justify your answer.
- Determine the largest q for which such an identity *would* beat Theorem 23.13.3.

Hint: compare logarithms to a common base.

Solution.

- (a) One level replaces a product of size $3m$ by q products of size m , so by induction on p the recursion performs q^p multiplications at size $n = 3^p$, the case $p = 0$ being one multiplication. The additions at one level cost Cm^2 for a constant C depending only on the identity, so the operation count satisfies $T(3m) = qT(m) + Cm^2$ with $T(1) = 1$. Unrolling,

$$T(3^p) = q^p + C \sum_{j=0}^{p-1} q^j 9^{p-1-j} = q^p + C 9^{p-1} \frac{(q/9)^p - 1}{(q/9) - 1} \leq q^p \left(1 + \frac{C}{q-9}\right),$$

where the last step uses $q > 9$ and drops the -1 . So $T(n) \leq cn^{\log_3 q}$ with $c = 1 + C/(q-9)$, since $q^p = n^{\log_3 q}$. Padding to a power of three as in [Theorem 23.13.3](#) extends this to all n , so $\omega \leq \log_3 q$.

- (b) No. $\log_3 23 = \ln 23 / \ln 3 = 3.13549 \dots / 1.09861 \dots = 2.85404 \dots$, while $\log_2 7 = 2.80735 \dots$. Since $2.854 > 2.807$, the exponent obtained from 23 products at size 3 is worse than Strassen's.
- (c) We need $\log_3 q < \log_2 7$, that is $q < 3^{\log_2 7}$. Now $3^{\log_2 7} = e^{(\log_2 7) \ln 3} = e^{3.08398 \dots} = 21.8498 \dots$, so the condition is $q \leq 21$. Indeed $\log_3 21 = 3.04452 \dots / 1.09861 \dots = 2.77124 \dots < \log_2 7$, while $\log_3 22 = 3.09104 \dots / 1.09861 \dots = 2.81359 \dots > \log_2 7$. So an identity with 21 multiplications would beat Strassen and one with 22 would not.

Exercise. C2: Where the entrywise bound really fails

Let $\varepsilon > 0$ and $\mathbf{A} = \mathbf{B} = \begin{pmatrix} 1 & \varepsilon \\ \varepsilon & \varepsilon \end{pmatrix}$.

- (a) Compute $\mathbf{AB}$ and $|\mathbf{A}||\mathbf{B}|$ exactly, and identify the entry of $\mathbf{AB}$ that is smallest as $\varepsilon \rightarrow 0$.
- (b) Compute $\mathbf{P}_1, \dots, \mathbf{P}_7$ exactly, and find which of them is largest as $\varepsilon \rightarrow 0$.
- (c) Suppose $\varepsilon, 2\varepsilon, 1-\varepsilon, 1+\varepsilon, 1+2\varepsilon$ and $2\varepsilon^2$ are representable, and that the model rounds $(1+\varepsilon)^2$ to $1+2\varepsilon$ and $(\varepsilon-1)(1+\varepsilon)$ to -1 , each time losing a trailing ε^2 . Determine $\hat{c}_{22}$, and the least constant c for which $|\hat{\mathbf{C}} - \mathbf{AB}| \leq cu|\mathbf{A}||\mathbf{B}|$ can hold at the $(2, 2)$ position.

Hint: with $\varepsilon = 2^{-27}$ in 53-bit binary arithmetic, both roundings are what rounding to nearest does.

Solution.

- (a) $\mathbf{AB} = \begin{pmatrix} 1+\varepsilon^2 & \varepsilon+\varepsilon^2 \\ \varepsilon+\varepsilon^2 & 2\varepsilon^2 \end{pmatrix}$, and since every entry of $\mathbf{A}$ and $\mathbf{B}$ is positive, $|\mathbf{A}||\mathbf{B}| = \mathbf{AB}$.

The smallest entry as $\varepsilon \rightarrow 0$ is the $(2, 2)$ entry $2\varepsilon^2$.

- (b) With $a_{11} = b_{11} = 1$ and all other entries ε ,

$$\begin{aligned} \mathbf{P}_1 &= (1+\varepsilon)^2, & \mathbf{P}_2 &= 2\varepsilon, & \mathbf{P}_3 &= 0, & \mathbf{P}_4 &= \varepsilon^2 - \varepsilon, \\ \mathbf{P}_5 &= \varepsilon + \varepsilon^2, & \mathbf{P}_6 &= \varepsilon^2 - 1, & \mathbf{P}_7 &= 0. \end{aligned}$$

As $\varepsilon \rightarrow 0$ the largest in modulus is $\mathbf{P}_1 \rightarrow 1$, with $\mathbf{P}_6 \rightarrow -1$ beside it; the other five all tend to 0.

- (c) The $(2, 2)$ entry is assembled as $\mathbf{P}_1 + \mathbf{P}_3 - \mathbf{P}_2 + \mathbf{P}_6$, and by (b) this is $(1 + 2\varepsilon + \varepsilon^2) + 0 - 2\varepsilon + (\varepsilon^2 - 1) = 2\varepsilon^2$, as it must be. Two of the four terms have modulus close to 1, and the two occurrences of ε^2 that survive the cancellation are exactly the two the model throws away: every operand that enters the $(2, 2)$ entry — $a_{11} + a_{22} = b_{11} + b_{22} = b_{11} + b_{12} = 1 + \varepsilon$, $a_{21} + a_{22} = 2\varepsilon$, $b_{12} - b_{22} = 0$ and $a_{21} - a_{11} = \varepsilon - 1$ — are all representable, so each is formed exactly, and then by hypothesis $\hat{\mathbf{P}}_1 = 1 + 2\varepsilon$ and $\hat{\mathbf{P}}_6 = -1$, while $\hat{\mathbf{P}}_3 = 0$ and $\hat{\mathbf{P}}_2 = 2\varepsilon$ are exact. Assembling left to right,

$$\text{fl}((1 + 2\varepsilon) + 0) = 1 + 2\varepsilon, \quad \text{fl}((1 + 2\varepsilon) - 2\varepsilon) = 1, \quad \text{fl}(1 + (-1)) = 0,$$

so $\hat{c}_{22} = 0$ and the error is $2\varepsilon^2$. Since $(\|\mathbf{A}\|\|\mathbf{B}\|)_{22} = 2\varepsilon^2$ by (a), an entrywise bound there reads $2\varepsilon^2 \leq cu \cdot 2\varepsilon^2$, so the least admissible constant is $c = 1/u$, exactly as in [Theorem 23.13.6](#). The mechanism is the same: a quantity of size ε^2 is carried inside numbers of size 1, where the model cannot see it.

Exercise. C3: Only the products recurse

Suppose someone proposes an identity that multiplies 2×2 matrices with eight multiplications but only two additions, and recurses on it.

- Write down the recurrence for its operation count with $T(1) = 1$, and solve it exactly for $n = 2^p$.
- Hence determine the exponent it gives, and compare with [Theorem 23.13.3](#).
- Explain in one sentence what part of an identity the exponent depends on.

Solution.

- Two block additions of size $m \times m$ cost $2m^2$, so $T(2m) = 8T(m) + 2m^2$ with $T(1) = 1$. We claim $T(2^p) = \frac{3}{2} \cdot 8^p - \frac{1}{2} \cdot 4^p$. For $p = 0$ this is $\frac{3}{2} - \frac{1}{2} = 1$. Assuming it for p and taking $m = 2^p$, so $m^2 = 4^p$,

$$T(2^{p+1}) = 8\left(\frac{3}{2}8^p - \frac{1}{2}4^p\right) + 2 \cdot 4^p = \frac{3}{2}8^{p+1} - 4 \cdot 4^p + 2 \cdot 4^p = \frac{3}{2}8^{p+1} - \frac{1}{2}4^{p+1},$$

since $-4 + 2 = -2$ and $-2 \cdot 4^p = -\frac{1}{2}4^{p+1}$. (Check: $T(2) = 10$, $T(4) = 88$, $T(8) = 736$.)

- With $n = 2^p$ this is $T(n) = \frac{3}{2}n^3 - \frac{1}{2}n^2$, so the exponent is 3. Saving additions changed the constant from 2 to $\frac{3}{2}$ and nothing else; [Theorem 23.13.3](#) changed the exponent from 3 to $\log_2 7$.
- Only the number of *multiplications* recurses, so only that number enters the exponent; the additions contribute to the constant.